

Complete-Cost Expert Fusion for Verified Repeated Numerical Solves

Author Names to Be Added

Abstract—Repeated numerical solves span scientific computing, yet no solver is uniformly best. Learned candidates can lose their advantage after correction, verification, and continuation. We present SMAVE, an Equation-Mixture-of-Experts runtime minimizing reach-weighted complete cost across classical and learned experts, correction, original-equation gates, and terminal fallback. For order-invariant cascades, we derive the optimal cost-per-acceptance order. Exact dynamic programs jointly select expert, correction budget, and order under independent or adjacent-transition costs; the interaction-aware decision form is NP-complete. Every returned result must pass an original-equation gate.

On one Apple M4, a frozen 19-action policy evaluated once on three untouched SuiteSparse matrices and 24 right-hand-side/tolerance requests reaches $1.000030794 \times$ realized-oracle regret, versus $1.001 \times$ for fixed SuperLU and $1.756 \times$ for a static profile. All requests succeed with zero gate, plan-order, or exact-DP mismatch. With no family adaptation or interaction transition, this is near-oracle control selection, not request adaptation. A zero-execution replay on frozen v5 switches 0/24 requests and remains $1.718 \times$ worse than global fixed, so the guard is not a general repair. Ten-worker gate-only speedups reach $3.51 \times / 2.40 \times$; Navier–Stokes gate and complete-path scaling reach $2.07 \times / 1.21 \times$. Seven PDEBench-derived comparisons and HINTS provide qualified evidence. Results remain one-host and workload-specific, not universal or cross-platform acceleration.

Index Terms—algorithm selection, heterogeneous computing, numerical solvers, neural operators, parallel verification, scientific machine learning, solver portfolios

1 INTRODUCTION

Large simulation, optimization, and control pipelines rarely solve one equation instance in isolation. They repeatedly solve related sparse linear systems, nonlinear residual equations, ordinary differential equations (ODEs), differential-algebraic equations (DAEs), and event-driven subsystems as parameters, states, or boundary conditions change. Mature numerical libraries provide robust general-purpose algorithms for these families [1]–[4]. However, the best algorithm and hardware path depend on structure, scale, conditioning, reuse, and requested accuracy. A single globally fixed solver consequently leaves substantial performance on the table.

Learned numerical components create a second opportunity and a second failure mode. Neural operators and learned preconditioners can amortize computation over a

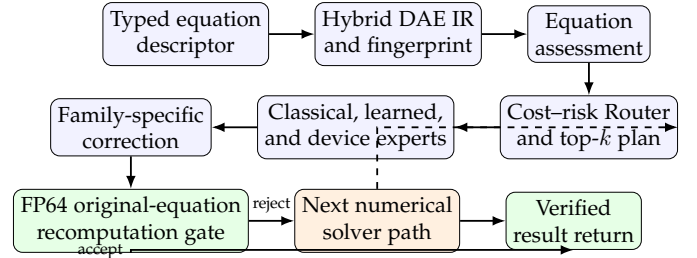


Fig. 1. SMAVE’s verification-aware expert-fusion pipeline. Dashed control flow returns to the plan when an expert is inapplicable; no candidate bypasses the family-specific original-equation gate.

parameterized family [5]–[7], but raw prediction time is not complete solve time. Candidate construction, data movement, local correction, original-equation verification, and the next numerical attempt after rejecting an unsuitable candidate can dominate short solves. Moreover, a low average test error does not establish that an individual candidate satisfies the original equation. Directly replacing a classical solver therefore couples performance uncertainty to numerical risk.

This paper studies the following scientific question:

Can heterogeneous classical and learned solver experts be fused so that selection minimizes complete verified runtime, while the original equations—not the learned model or router—determine which result is returned?

This question differs from conventional and numerical solver selection [8]–[10] in two ways. First, the selected object is not a single solver but a pipeline consisting of candidate generation, optional correction, a separately evaluated family-specific gate, and numerical continuation after rejection. Second, selection quality is measured by end-to-end verified cost and numerical rejection/continuation behavior, not raw kernel latency or hindsight winner accuracy.

We answer through SMAVE, a verification-aware Equation-Mixture-of-Experts runtime. It lowers typed equations to a common IR and constructs top- k plans over classical, learned, and device experts with explicit candidate, corrector, verifier, and numerical continuation roles. A candidate is returned only after an FP64 gate evaluates the original residual, constraints, defect, or event consistency; rejection restarts the next numerical path from the original state.

Our contributions are:

- 1) We formulate heterogeneous numerical acceleration as a *verification-aware expert-selection* problem and derive a reach-weighted complete-cost objective for candidate cascades, correction, original-equation verification, transfer, rejection, and the terminal classical solve. For a fixed eligible cascade with order-invariant stage statistics, we apply the classical pairwise-optimal cost-per-acceptance ordering rule. For finite calibrated correction actions, we prove exact dynamic programs for independent and adjacent-transition-cost models under a one-budget-per-expert constraint. We also prove the interaction-aware decision problem NP-complete, explaining the exponential dependence, and verify both optimizers against exhaustive finite oracles.
- 2) We define a role-constrained candidate–corrector–gate–continuation pipeline in which every returned result must satisfy a family-specific original-equation contract. This separates numerical acceptance from router and learned-model quality: a poor expert may waste time but cannot bypass the gate.
- 3) We implement the abstraction across algebraic and dynamic solver interfaces, while separating interface coverage from acceleration evidence.
- 4) We retain complete-cost gains and failures. Seven PDEBench-derived workloads span $1.58 \times -135.14 \times$ median speedup with all 95% lower bounds above one; two operator studies beat a shared hybrid control; and an official-code HINTS comparison uses the same equation contract. Gate-only, complete-path, device, transfer, and nonlinear-routing negatives remain visible. Final public routing is mixed: v6 reaches $1.000030794 \times$ oracle regret, only 0.109% below fixed, whereas the frozen v5 replay remains $1.718 \times$ fixed.

The results support a bounded conclusion: verified expert fusion can accelerate qualified repeated workloads, but it is not a universal replacement for classical solvers. Verification cost, model applicability, correction structure, and hardware placement must be treated as first-class design variables.

2 BACKGROUND AND RELATED WORK

2.1 Classical Solver Ecosystems

Sparse direct and Krylov methods remain the correctness and robustness foundation for numerical simulation. GMRES handles nonsymmetric systems [1]; SuperLU exposes sparse LU factorizations [2]. SUNDIALS provides nonlinear, ODE, and DAE solvers with mature integration contracts [3], while PETSc organizes portable solver composition [11]. These systems demonstrate the value of composable backends, but they generally require a user or fixed policy to choose among algorithms. SMAVE builds on them as authoritative experts rather than attempting to replace their numerical semantics.

2.2 Learned PDE and Equation Solvers

DeepONet and Fourier neural operators learn mappings between function spaces [5], [6]. PDEBench provides a

multi-family benchmark for scientific machine learning [12]. These approaches can produce very fast inference for fixed families, but their reported error and speedup contracts vary in predicted quantity, hardware, training cost, and whether the original equation is checked per request.

A complementary line learns components inside an iterative method. Learned preconditioners can reduce conjugate-gradient time while the iterative residual remains authoritative [7]. FCG-NO places a resolution-transferable neural operator inside flexible conjugate gradients as a nonlinear preconditioner [13]. SMAVE generalizes this principle to a portfolio: learned warm starts, inverse actions, multigrid cycles, and full-state operators may all propose candidates, but their role and correction path are decided per equation family.

Recent hybrid methods make the comparison closer. HINTS interleaves a DeepONet with relaxation to balance error modes in an iterative PDE solver [14]. The Greedy PDE Router chooses neural or classical updates at each iteration to optimize the error trajectory [15]. Error-Conditioned Neural Solvers instead feed the residual field to a learned corrector and note that small residual need not imply small reconstruction error in ill-conditioned systems [16]. SMAVE asks which typed pipeline remains profitable after every stage and rejection is charged. Its gate is mandatory final acceptance under the declared equation and constraint tolerance, not a proxy for unavailable ground-truth error.

2.3 Algorithm Portfolios and Mixtures of Experts

Rice’s algorithm-selection formulation maps problem features to algorithms and performance [8]; later surveys identify feature cost, distribution shift, and runtime overhead as central concerns [9].

Learned numerical-solver selection is also established. Lighthouse combines solver taxonomies with performance classification [10]. Recent work adds matrix/right-hand-side representations [17], run-local adaptation [18], and regret guarantees for sequential SOR tuning [19]. We therefore claim neither request conditioning, unseen-matrix prediction, nor online solver adaptation as unique.

Cost-per-success ordering is also prior art: independent early-stop tests admit ratio orderings, including extensions with precedence constraints [20]. We use that ordering as a lemma rather than a novelty claim; Proposition 1 instead states the exactness boundary of our finite one-budget-per-expert recurrence.

Multimode diagnosis already uses dynamic programming when mode changes alter later test costs [21]. Our model keeps only adjacent cost effects and history-independent acceptance; neither transition-aware sequencing nor DP is new.

Numerical solving introduces an additional constraint: a router decision is not itself evidence that a candidate satisfies the equation. SMAVE therefore uses “gate” in the numerical sense of an acceptance test computed from the original equation, distinct from the learned routing gate in a conventional MoE. The router may be wrong without making the returned answer wrong; a bad decision increases cost or continues to another numerical path, because the original-equation gate determines acceptance.

2.4 Positioning

HINTS and the Greedy PDE Router contribute iteration-level hybrids; solver selectors contribute request features and adaptation. SMAVE instead studies a typed request-level cascade, a reach-weighted objective charging rejection and terminal fallback, and a return invariant that routing cannot bypass original-equation acceptance. The dated search audit supports no “first” claim for this combination or for the underlying ratio ordering.

3 VERIFICATION-AWARE SOLVER SELECTION

3.1 Equation Workload

Let $q \in \mathcal{Q}$ denote one solve request. A request contains an equation family, dimensions and sparsity, coefficients or callback handles, initial state, tolerances, available compute backends, and an original state s_q . The equation family defines an authoritative residual or defect operator $\mathcal{R}_q(y)$ and a scaled acceptance threshold τ_q . For dynamic systems, y also contains time, derivatives, mode, and event metadata.

An expert $e \in \mathcal{E}$ may be a complete classical solver, a preconditioner, a warm start, a learned operator, a device kernel, a corrector, or a verifier. Its output is therefore typed by a role $\rho(e)$ rather than assumed to be a final solution. A plan

$$P = (e_{1:k}, c, g, t) \quad (1)$$

contains an ordered candidate cascade $e_{1:k}$, optional corrector c , mandatory gate g , and terminal numerical solver t . A gate is a numerical acceptance test derived from the original equation. If a stage is rejected, execution continues from the original request state to the next numerical path.

3.2 Complete Verified Cost

Let R_j denote the event that cascade stage j is reached and let A_j denote acceptance at that stage. Thus R_1 always occurs and $R_{j+1} = R_j \cap \neg A_j$. For stage j , let K_j include candidate, transfer, correction, and gate cost; let C_0 include assessment, routing, and shared setup; and let C_t include the terminal solver and its final gate. The conditional complete-cost model is

$$\begin{aligned} \mathbb{E}[C(P, q) \mid \phi(q)] &= C_0(q) + \sum_{j=1}^k \Pr(R_j \mid \phi(q)) \bar{K}_j(q) \\ &\quad + \Pr(R_{k+1} \mid \phi(q)) \bar{C}_t(q), \end{aligned} \quad (2)$$

where $\bar{K}_j = \mathbb{E}[K_j \mid R_j, \phi(q)]$ and $\bar{C}_t = \mathbb{E}[C_t \mid R_{k+1}, \phi(q)]$. This reach-weighted form makes rejection and numerical continuation explicit. A microsecond candidate is not beneficial if transfer, correction, verification, or its probability of reaching the terminal solver dominates the saved classical work.

The ideal router seeks

$$P^*(q) = \arg \min_{P \in \mathcal{P}(q)} \mathbb{E}[C(P, q) \mid \phi(q)] \quad (3)$$

subject to capability, memory, artifact, and role constraints. Here $\phi(q)$ is a feature vector derived without running all candidate experts. A hindsight per-scenario oracle is used only as an analysis reference; its search cost and query winner labels are excluded from deployable routing. In the

evaluated profiles, stage costs and reach frequencies come from disjoint training/calibration scenarios, either as action-level constants or as functions of $\phi(q)$; held-out outcomes are not available to the online decision.

The objective also yields an inspectable ordering rule. Let $p_j = \Pr(A_j \mid R_j, \phi(q)) > 0$ and assume that two eligible stages have order-invariant conditional costs and acceptance probabilities. Placing stage a before stage b costs $\bar{K}_a + (1 - p_a)\bar{K}_b$ before their common suffix; the reverse order costs $\bar{K}_b + (1 - p_b)\bar{K}_a$. Therefore, a should precede b if and only if

$$\frac{\bar{K}_a}{p_a} \leq \frac{\bar{K}_b}{p_b}. \quad (4)$$

Repeated adjacent exchanges show that sorting by $\bar{K}_j/p_j$ minimizes the expected cost of a fixed eligible cascade under these assumptions, consistent with prior independent-test ordering results [20]. This result does not choose the globally optimal subset when stages change one another’s state or when a contract fixes terminal-stage order.

Correction effort is itself a routing decision. Let $\mathcal{B}_e$ be the calibrated budgets available for expert e , and let action $a = (e, b)$ have conditional attempt cost K_a and acceptance probability p_a . At most one budget may be selected for each expert. After sorting actions by K_a/p_a , the optimal suffix value for action index i , used-expert set S , and remaining capacity r is

$$\begin{aligned} V(i, S, r) &= \min\{V(i+1, S, r), \\ &\quad K_i + (1 - p_i)V(i+1, S \cup \{e_i\}, r-1)\}, \end{aligned} \quad (5)$$

with the take branch disabled when $e_i \in S$, and terminal value C_t when $r = 0$ or no action remains. The exact dynamic program jointly chooses expert, budget, and order under a bounded action model. The implementation accepts either static calibrated (K_a, p_a) or request-conditioned predictions $(\hat{K}_a(\phi), \hat{p}_a(\phi))$; the original-equation gate remains outside the optimizer.

For adjacent transition effects, let $M_{b,a} > 0$ multiply K_a after action b , with $M_{\perp,a} = 1$. For used experts S , define

$$\begin{aligned} W(S, b) &= \min\{C_t, \min_{a: e_a \notin S} \{K_a M_{b,a} + \\ &\quad (1 - p_a)W(S \cup \{e_a\}, a)\}\}, \end{aligned} \quad (6)$$

with the inner minimum disabled when $|S| = k$.

Proposition 1 (Finite cascade exactness and hardness). *With finite nonnegative costs, positive history-independent acceptance, capacity k , and one action per expert: (i) when $M_{b,a} = 1$, sorting by K_a/p_a and Eq. (5) are globally exact in $O(n2^m)$ worst-case scalar states/transitions; (ii) with immediately previous-action multipliers, $W(\emptyset, \perp)$ is globally exact with $O(n^2 2^m)$ transitions; and (iii) the rational interaction-aware decision problem is NP-complete even for $K_a = 1$, $p_a = 1/2$, $k = n$, $C_t = 5$, and $M_{b,a} \in \{1, 2\}$.*

Proof. Adjacent exchange plus backward induction proves (i); Bellman induction on remaining experts proves (ii). For (iii), reduce directed Hamiltonian path with one action per vertex and multipliers one on edges and two otherwise. Appending an action costs at most $4.5 < C_t$, so every optimum uses all vertices, and cost is at most $\sum_{j=0}^{n-1} 2^{-j} + 5 \cdot 2^{-n}$ exactly for a Hamiltonian path. Membership in NP follows

by evaluating a cascade; complete details are in the supplementary artifact. $\square$

The 64-bit implementations support at most 63 experts and reject on state-cap overflow; exactness requires the cap to admit every reached state.

3.3 Numerical Acceptance and Continuation

The numerical contract is

$$\begin{aligned} \text{return}(y) &\implies g_q(y) = \text{accept}, \\ g_q(y) &\equiv \|D_q^{-1}\mathcal{R}_q(y)\|_\infty \leq \tau_q. \end{aligned} \quad (7)$$

with family-specific additions for backward error, integration defect, constraints, complementarity, event guards, and DAE hidden residuals. D_q contains absolute, relative, and nominal scaling. The gate is computed from the original equation rather than a router score or learned confidence and is re-evaluated after correction.

If e_i fails, produces non-finite values, violates a contract, or is rejected by g_q , the next candidate starts from the original request state. A dynamic step is returned only after all state, derivative, event, and mode checks pass. Thus selection error and candidate error affect runtime and solve completion, but not the acceptance rule.

Proposition 2 (Verified-result return). *Assume that every result-return path invokes g_q on the original request contract and returns only on acceptance. Then any successfully returned state y satisfies $g_q(y) = \text{accept}$, independently of the router ordering and of earlier candidate, corrector, or device failures.*

Proof. Each candidate path either fails, is rejected by g_q , or returns after acceptance. The first two cases proceed to another stage and cannot produce a successful result. The terminal solver is subject to the same rule. Therefore every successful return has passed g_q . $\square$

Proposition 2 is a numerical solver control-flow invariant. It makes the acceptance boundary explicit but is not a theorem about an incorrectly specified residual callback or faulty arithmetic hardware.

3.4 Claims and Scope

The formulation supports three testable claims. First, complete-cost selection can outperform a fixed expert on qualified workloads. Second, correction can make an otherwise unusable learned candidate profitable without allowing model output to bypass original-equation acceptance. Third, mandatory original-equation gates reject observed bad candidates and preserve the verified-result return invariant, while the experiments test the implemented gates and continuation paths on observed workloads. The claim does not extend to arbitrary incorrect callbacks or floating-point hardware faults. These claims concern solver-internal numerical mechanisms and their observed experimental evidence.

4 SYSTEM DESIGN

4.1 Typed Hybrid DAE Intermediate Representation

SMAVE accepts source-model front ends and a public typed ABI. Both lower to a Hybrid DAE IR containing variables, residual equations, block incidence, canonical CSR Jacobian structure, events, modes, resets, sample clocks, solver hints, and source hashes. Linear algebraic blocks, nonlinear systems, explicit ODEs, fully implicit DAEs, complementarity systems, and multi-rate block graphs are represented as specializations rather than unrelated solver paths.

This common representation serves two purposes. It exposes comparable features to the router, and it ensures that front ends, the command-line interface, and the shared-library SDK reach the same assessment, plan, expert, gate, and continuation implementations. Unsupported semantics remain explicit capability failures; they are not converted to an unverified text-based path.

4.2 Equation Assessment

Assessment computes a read-only feature vector before expensive candidate execution. Features include dimension, nonzero count, density, symmetry and positive-definiteness probes, block/SCC structure, differentiation index metadata, event topology, parameter-domain distance, artifact compatibility, historical expert cost, memory estimate, and available devices. Structural tests are allowed to reject an expert but cannot certify the final numerical result.

The assessment emits a family label and a capability-filtered expert set. For example, an SPD linear block may admit PCG with IC(0), Jacobi, or a learned multigrid preconditioner; a nonsymmetric block admits restarted GMRES with ILUT or ILU(0), sparse QR/LU, and a pivoting terminal solver. A nonlinear block admits a warm start or operator candidate followed by Newton correction. An eventful DAE requires state/derivative consistency projection, excluding the event-free caller stepper interface.

4.3 Expert Registry and Roles

Each expert declares its equation family, role, precision, device, memory model, artifact schema, cancellation boundaries, and continuation relationships. The registry distinguishes four roles:

- *candidate experts* produce a state, inverse action, or step proposal;
- *correctors* reduce the original residual from that proposal;
- *verifiers* evaluate original-equation acceptance contracts;
- *terminal experts* restart from the original request state.

Role separation avoids a common ambiguity in learned solvers: a fast predictor is not labeled a solver unless its downstream pipeline reaches an accepted result.

4.4 Two-Level Router

The first routing level filters experts by hard contracts. Before appending contract-required direct stages, the second orders remaining stages by the complete-cost index

in Eq. (4). The implementation uses a conservative stage-cost surrogate that adds a calibrated risk/OOD penalty before division by predicted acceptance probability; risk is therefore not a lexicographic override of complete cost. Contract-required direct and terminal solvers remain after speculative candidates. The current research baseline includes fixed rules, calibrated profiles, leave-one-scenario-out nearest neighbor and CART selectors, and a structure-derived cost selector. Exact winner accuracy is telemetry; practical regret relative to the gate-passing hindsight oracle is the primary measure because multiple experts may be equivalent within timing noise.

Legacy calibrated profiles use the ceiling of observed median iterations from fully gate-verified attempts. Multi-budget profiles instead expose calibrated (e, b) actions. A request-conditioned profile standardizes declared equation/context features, predicts log attempt cost with ridge regression and acceptance with regularized logistic regression, and attaches disjoint-calibration error to action risk. Without calibrated interactions, the Router applies Eq. (5). With conservative adjacent transition-cost multipliers, it applies Eq. (6) and explores order inside the exact finite recurrence. Both paths have an explicit state limit, select at most one budget per expert, and charge terminal continuation after rejection.

For the finite modeled action set, the dynamic programs return exact top- k cascades under either independent costs or immediately previous-action cost multipliers when the state cap admits all reachable states. The 64-bit mask permits at most 63 experts; cap overflow is an error, not an approximate plan. Exactness does not extend to longer-memory costs or history-dependent acceptance. Built-in experts execute before a caller-supplied terminal solver. Learned artifacts are content-addressed and bound to equation family, source fingerprint, dimensions, precision, and training domain. A mismatched, revoked, or malformed artifact is excluded before inference. Proposition 1 gives the exactness, complexity, and hardness boundaries.

4.5 Heterogeneous Placement

Requests with the same expert version, fingerprint, shape, precision, tolerance class, and mode are bucketed for batched execution. CPU SIMD is preferred for small micro-solves unless expected reuse exceeds device setup and transfer costs. Metal GPU and Apple Neural Engine paths generate FP32 or device-specific candidates, which are independently refined and verified on CPU FP64. Device placement is therefore a routing decision, not a correctness decision.

4.6 Public Runtime Boundary

The implementation exports opaque library, problem, solver, cancellation, and result handles through a frozen C ABI. Descriptors carry size and ABI version; callbacks provide structured residuals, Jacobians, guards, and resets. Relative deadlines and cancellation are cooperative. A bounded thread-local, per-library error stack reports status, operation, message, and trace identifier for covered pre-result failures. These facilities make experiments reproducible from an installed host without exposing C++ implementation types.

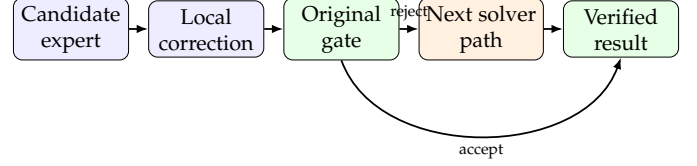


Fig. 2. A fused solve is a typed numerical pipeline, not an averaged prediction. Each continuation path restarts from the original state and problem data.

5 VERIFICATION-AWARE EXPERT FUSION

5.1 Candidate–Corrector–Gate Pipeline

Figure 2 shows the execution pipeline. A candidate receives the original request state. If it returns a finite proposal, a family-specific corrector may use that proposal as an initial condition. The gate then evaluates the original problem representation, not the candidate’s internal loss, reduced model, or approximate Jacobian. Acceptance returns the verified result; rejection invokes another expert or the terminal classical solver from the original state. This control flow implements Proposition 2: every successful return passes the original-equation gate. Candidate diagnostics remain observable, but an unverified candidate is never labeled a successful solve.

5.2 Linear and Nonlinear Algebraic Gates

For $Ax = b$, SMAVE evaluates a scaled residual and norm-wise backward error using the original matrix representation. Dense and CSR layouts share the same numerical contract. Learned inverse actions and preconditioners cannot directly return a solution; a Krylov or correction stage and the original-equation gate determine acceptance.

For nonlinear $F(x) = 0$, the candidate is copied into a fresh Newton state. The corrector may use a callback Jacobian, finite-difference Jacobian, learned preconditioner, or family-specific structure. The final $F(x)$ is evaluated again through the original callback. An incorrect candidate Jacobian can affect cost but cannot replace the final residual.

5.3 ODE and DAE Gates

Explicit ODE candidates are checked for finite RHS evaluations and embedded or dense-output integration defects. The external event-free stepper interface returns quarter, midpoint, three-quarter, and endpoint states; the runtime independently evaluates the original RHS at five nodes and checks two composite-Simpson defects.

For a fully implicit DAE $F(t, y, \dot{y}) = 0$, the gate checks the original residual and backward-Euler kinematics. When an index-2 Hessenberg-like contract applies, it also checks hidden-constraint residual and hidden-Jacobian rank. Event solves locate roots, apply ordered reset batches, project consistency, and re-evaluate guards before returning state, derivative, mode, and audit metadata.

5.4 Complementarity and Block-Graph Gates

Strongly monotone linear complementarity candidates are checked for primal and dual feasibility and complementarity product. Multi-rate block graphs independently re-evaluate callback outputs, connection values, and same-rate

fixed-point residuals. A local node continuation result is accepted only if its original output gate passes, preventing one component from bypassing graph-level correctness.

5.5 Strict Gate Optimization

The default contract performs a per-request FP64 original-expression gate. SMAVE also evaluates a fused strict kernel that reorganizes the same original-equation arithmetic without changing decisions or tolerances.

5.6 Failure and Cancellation Semantics

Cancellation and relative deadlines are observed at cooperative boundaries between candidate attempts, nonlinear iterations, integration steps, root localization, and event publication. User callbacks and third-party kernels are not forcibly preempted. Cancellation or callback failure returns a stable failure status rather than a partial candidate; the next numerical path restarts from the original request state.

5.7 Numerical Failure Probes

Table 1 maps the verified-result return rule to implementation-level numerical failure probes. The table is not a formal verification of the C++ implementation; it records which candidate, gate, and continuation boundaries are exercised.

6 EXPERIMENTAL METHODOLOGY

6.1 Research Questions

The evaluation addresses five research questions.

- RQ1: Does one verified runtime cover heterogeneous equation families without hiding failures or unsupported comparisons?
- RQ2: When does complete-cost expert fusion accelerate repeated solves?
- RQ3: Does calibrated routing improve over a fixed expert, jointly select correction budget, and remain effective under workload shift?
- RQ4: How do correction and original-equation verification affect learned candidates?
- RQ5: What do complete-cost decomposition, strict-gate, parallelism, and device-placement studies reveal about the remaining bottlenecks?

6.2 Platform and Build

The authoritative results in this draft were generated on an Apple M4 ARM64 system running macOS 26.5.2, Apple Clang 21.0.0, CMake 4.3.2, and a Release build. CPU paths use FP64 unless a device candidate explicitly declares another precision; all final equation gates are evaluated with the path-specific published contract, and the primary runtime contract uses CPU FP64 original expressions. Because this is one machine and software stack, we treat performance portability as an open threat rather than extrapolating to Linux/x86-64 or CUDA.

6.3 Workloads

Table 2 summarizes the evidence suites.

The seven PDEBench-derived assets in Table 2 use designated classical baselines on identical inputs and accuracy contracts. Complete time includes setup, candidate, correction, and gate. Each workload has one warm-up and 30 process-level repetitions with alternating workload order and fixed inference threads. A separate probe alternates classical-first and SMAVE-first phase order over 30 matched pairs; run-level total-runtime ratios, not within-run solves, are the inference unit.

The linear and 16-variable nonlinear operator studies use 16 training and 64 disjoint held-out scenarios, repeated in 100 batches (6,400 Runtime requests). A nonsymmetric Helmholtz replication is retained as a negative transfer result.

Routing transfer uses a separate 5×5 family with 64 scenarios and 20 repetitions per axis: a source diagonal of 4.5, a conditioning shift to 3.5, and a topology shift with west/east coefficients 1.35/0.65. Production competition and family-router paths include all candidate, gate, trace, and continuation cost; metrics cover eligible-expert calibration/rank, regret, speedup, status changes, and misroutes.

Both primary operator families also use one shared transparent hybrid control. It reuses the same fitted FP64 latent candidate and held-out scenarios, applies at most 32 weighted diagonal-residual Jacobi corrections, invokes a strict original-equation acceptance gate, and sends every rejection to the original classical solver. Candidate inference, correction, both gates, trace persistence, and continuation are included in the paired complete-runtime measurement. This control is a common algorithmic baseline, not an implementation of an external published method.

The HINTS baseline pins official revision 0c8b712f81ed08bdf27c3a215f8edb99910f5e2f [14], pretrained weights, all 750 Poisson cases, 400 updates, the 25:1 schedule, and $\omega = 2/3$. Both methods solve the exported 29-unknown system under the same FP64 normalized residual. Five runs follow three warm-ups; setup and HINTS diagnostics are excluded from online time. SMAVE uses its unmodified production path.

After headline timing, the production corrector is swept over $\{0, 1, 2, 4, 8, 16, 32\}$ for 30 rotated repetitions of all 64 held-out requests. Every point includes candidate, correction, gate, trace, and continuation; the stable mechanism result is the minimum budget attaining full acceptance, not the timing winner.

For joint routing, three calibrated actions span two nonlinear experts with $k = 2$. An independent enumerator checks every valid subset, budget, and order; production Runtime must reproduce the selected rejection-continuation-acceptance path. To test the finite-action recurrence beyond that probe, a fixed-seed property sweep constructs 256 six-action cases (three experts, two budgets each), cycles k from one to three, and exhaustively enumerates subsets and permutations. A second 256-case sweep adds 24 adjacent multipliers; all 4,096 directed four-vertex graphs check the hardness reduction. A prefrozen planning-only counter study spans up to 16 experts plus the deployed shape. For profile transfer, two 12-variable nonlinear SCC families each freeze Runtime-trace profiles on 32 scenarios

TABLE 1

Implementation probes for the verified-result return rule. Every probe is emitted as a machine-readable release report and retains rejection or failure as an explicit outcome.

Case	Injected numerical condition	Required observation	Release evidence
(P1)	External ODE/DAE stepper returns an inaccurate proposal; inaccurate nonlinear raw candidate enters the pipeline	The original-equation gate rejects the candidate; no successful result is reported	External-stepper continuation and nonlinear-operator reports
(P2)	Expert or bundle metadata is incompatible	Incompatible artifacts are excluded before candidate execution	Component-ablation report
(P3)	Cancellation or deadline fires at a cooperative solve boundary	Stable failure status is returned without a partial success	C API cancellation report
(P4)	External linear/ODE/DAE terminal solver fails, is cancelled, or violates a negative contract	The terminal solver runs after built-ins and must pass the original linear, dense-output, or DAE kinematic gate	External linear and stepper continuation reports

TABLE 2

Evaluation suites and the claims they support. Counts are retained even when SMAVE or the baseline does not produce a common successful result.

Suite	Cases	Equation scope	Primary use in this paper
SuiteSparse	39 matrices	Sparse linear systems	Common-success agreement, back-end coverage, and explicit no-common-success accounting
PETSc TS	17 equation cases	ODE/PDE time integration	Same-input trajectory and end-to-end comparisons against an external framework
OpenModelica MSL	7 models	Generated simulation systems	Embedded linear calls, trajectory agreement, and integration into a model runtime
COPS	68 instances	Nonlinear optimization and KKT systems	KKT agreement, external numerical continuation, resource gating, and full-NLP outcome classification
PDEBench-derived	7 families	Discretized PDE workloads	Authoritative verified speedup comparison under per-family same-input contracts
Operator studies	2 families	Repeated linear and nonlinear blocks	Held-out training/evaluation split, correction, gate, break-even, and complete runtime

and report regret against an exhaustive oracle on 32 disjoint held-out scenarios; this is not a workload-speedup measurement.

The request-conditioned extension has three 12-variable nonlinear families, three experts, four budgets each, and $k = 3$ (12 actions). Runtime traces provide 192/96/192 training/calibration/held-out requests with three repetitions per action. Fixed ridge and logistic models are evaluated by cost error, Brier/calibration error, realized per-request oracle regret, static/fixed controls, plan diversity, exact-DP agreement, and production gate/continuation outcomes; held-out data never enter fitting or calibration.

The public extension uses fixed six/four-matrix training/calibration sets, eight RHS/tolerance requests per matrix, and a 19-action PCG/GMRES/direct union. Three development folds establish the control-aware anchor rule without final-cohort data. Each final cohort contains three matrix-ID- and collection-group-disjoint matrices, one from each numeric class, and each compatible action is measured five times under a same-request counterbalanced terminal reference. Evaluation covers cost/pass prediction, exhaustive realized regret, control ablations, attempt order, independent original-matrix acceptance, terminal continuation, and action interaction.

The protocol preserves all final outcomes. V4 is contract-

invalid because its frozen verifier asserted 48 requests; v5 is a valid negative first run. After the guard and v6 package were frozen from development evidence, v6 was executed exactly once. A deterministic v5 counterfactual applies the current C++ anchor comparison to frozen tables with no solver execution. V5/v6 selection, payload, source, executable, verifier, and output-path contracts precede timing; no further cohort is searched. A post-hoc zero-execution audit uses held-out action statuses only for support-shift diagnosis.

The intra-node study reuses 40 Navier–Stokes Helmholtz solves at 1, 2, 4, 8, and 10 workers. Each point has one warm-up and 30 alternating-order repetitions; Accelerate uses one internal thread. Fixed-seed 10,000-resample intervals use paired speedup ratios.

The batch study fixes ten workers and increases same-dataset requests from 12 to 160 solves per process under the same protocol. Throughput is solves divided by complete verified time and normalized to 12 solves.

A second worker study isolates the fused gate on 2,080 linear and 2,080 nonlinear requests per repetition. It includes thread creation, alternates worker order over 30 repetitions, and compares every decision/residual with a sequential reference. It measures gate throughput, not complete-solve scaling.

6.4 Baselines and Comparisons

Classical baselines are workload-specific and include structured direct solvers, PCG with classical preconditioners, restarted GMRES with ILUT/ILU(0), sparse direct backends, finite-difference Newton, and original model solvers. For routing, we compare a fixed expert, the online rule, a calibrated profile, simple feature selectors, and a hindsight per-scenario best gate-passing expert. The hindsight oracle is not deployable: it excludes search cost and uses query outcomes, so it is reported only as a reference for regret. The final SuiteSparse hierarchy separately reports the raw conditioned model, training-family fixed anchor, current control-aware anchor, global fixed SuperLU, static profile, and realized per-request oracle. These names are not interchangeable: the current policy may abstain to global fixed, while an offline replay can test that decision rule without producing new timing observations.

6.5 Metrics and Statistical Contract

The primary performance metric is paired complete-runtime speedup

$$S = \text{median}_i \frac{T_{\text{baseline},i}}{T_{\text{SMAVE},i}}. \quad (8)$$

Where repeated paired samples are available, reports include a fixed-seed bootstrap 95% interval and paired win rate. A speedup claim is considered stable only when the interval lower bound exceeds one. We additionally report residual or QoI error, acceptance, continuation, failure, timeout, no-common-success, gate mismatch, training time, and break-even queries.

For the PDEBench-derived study, each paired sample is the baseline-to-SMAVEtotal runtime ratio from one measured process run. The reported median, fixed-seed 10,000-resample bootstrap interval, and run win rate are computed over 30 such ratios. The counterbalanced all-workload probe reports the paired ratio between SMAVE-first and classical-first speedups for each of seven workloads. We summarize the per-workload median ratio and fixed-seed 10,000-resample paired bootstrap interval; this quantifies order sensitivity under two complete phase orders rather than proving order invariance. No per-solve independence is assumed. Router, operator, and gate studies likewise retain their declared paired repetition units and use the interval rule above.

For a run with zero observed erroneous accepts in n independent trials, the one-sided exact upper bound at confidence $1 - \alpha$ is

$$p_{\text{upper}} = 1 - \alpha^{1/n}. \quad (9)$$

The operator validations count 64 independent held-out scenarios, not 6,400 timing repetitions, and hence report a 95% upper bound of 4.57% after zero observed erroneous accepts. This conservative scenario-level statistic is separate from the deterministic residual decision for each candidate and is not a claim of zero risk.

6.6 Reproducibility and Claim Extraction

Every manuscript value is tied to a machine-generated report. The normalized core archive rebuilds Release, passes

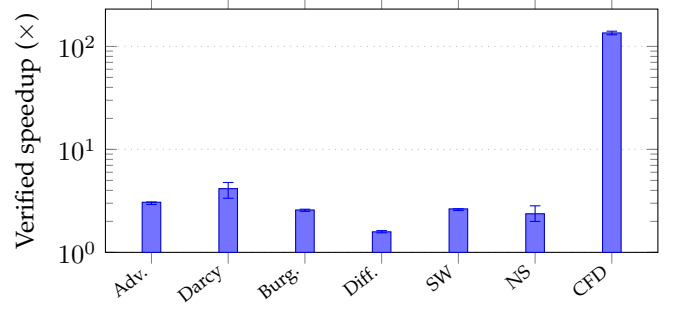


Fig. 3. Verified end-to-end speedup over each workload’s designated classical baseline. The log axis exposes both the six low-single-digit improvements and the single 135.14 $\times$ result; error bars are bootstrap 95% intervals over 30 run-level paired ratios.

29/29 tests, reruns solver evidence checks, and rebuilds the PDF; large benchmark payloads remain externally acquired.

7 EVALUATION

7.1 RQ1: Breadth, Agreement, and Explicit Failure Accounting

Table 3 reports cross-suite outcomes. SMAVEagrees on 31 of 39 SuiteSparse matrices; all eight exceptions remain. All 17 PETSc cases complete, seven OpenModelica models execute 535,728 embedded linear calls, and all 68 COPS instances reach KKT comparison. Full-NLP coverage is narrower because resource gates, terminal-only cases, and timeouts are retained.

These counts demonstrate integration breadth, not universal solver dominance. In particular, a SuiteSparse failure and a COPS terminal-only solve remain failures of the stronger performance claim even if the runtime rejects the unsuccessful path.

7.2 RQ2: Verified PDE Workload Acceleration

Figure 3 and Table 4 report 30 paired runs per workload. Medians span $1.58\times$ – $135.14\times$; every 95% lower bound exceeds one, and every workload wins at least 29 runs.

Six workloads lie below $4.16\times$; only CFD1D exceeds $100\times$, rejecting a universal two-order-of-magnitude claim. In the 30-pair counterbalanced study, order ratios span 0.991 – 1.040 (maximum shift 4.0%). 6 of 7 intervals include one; Diffusion–Sorption alone shows a small nonzero effect, while all 420 runs remain faster than baseline.

The mechanisms also differ. Advection and Burgers profit from equation-structured batched recurrences and vectorized gates. Shallow Water and incompressible Navier–Stokes use learned/structured transforms with independent residual checks. The CFD workload has unusually favorable repeated structure and amortization. This heterogeneity motivates routing by equation family and complete cost rather than a single globally advertised expert.

7.3 RQ3: Routing Versus Fixed and Hindsight Experts

On the evaluated sparse family, the calibrated router selects SuperLU instead of a fixed structured tridiagonal expert and achieves a paired median speedup of $5.077\times$ with a

TABLE 3

Cross-suite coverage from the authoritative overall report. “Compared” means a same-input agreement or performance comparison exists under that suite’s contract.

Suite	Attempted	Compared	Explicit exception
SuiteSparse	39	31	8 no common success
PETSc TS equations	17	17	0 pending
OpenModelica MSL	7	7	1 no linear subsystem
COPS KKT	68	68	0 failed
COPS full NLP	68	12 perf.	46 terminal-only
PDEBench-derived	7	7	0 unreported

TABLE 4

PDEBench-derived same-input comparisons. Speedups are medians over 30 paired process runs; brackets show fixed-seed bootstrap 95% intervals. Each SMAVetime includes setup, candidate kernel, and original-equation gate.

Workload	Solves	Baseline	Result [95% CI]
Advection	150	upwind	$3.066 \times [2.919, 3.087]$
Darcy	3	Jacobi-PCG	$4.157 \times [3.353, 4.771]$
Burgers	150	scalar direct	$2.580 \times [2.502, 2.626]$
Diffusion-Sorption	150	tridiagonal	$1.583 \times [1.548, 1.627]$
Shallow Water	60	PCG	$2.643 \times [2.550, 2.653]$
Navier-Stokes	40	10-wkr PCG	$2.371 \times [1.998, 2.832]$
CFD1D	90	PCG	$135.138 \times [129.740, 140.494]$

bootstrap 95% interval of $[5.064, 5.088]$ and a 99.7% paired win rate. The maximum mixed QoI error is 1.75×10^{-6} , with no dangerous misroutes in the report.

The gate-passing hindsight reference reveals remaining headroom. The calibrated plan’s median cost is $1.076 \times$ the per-scenario hindsight oracle and lies within 5% of it on 41.3% of samples. A leave-one-scenario-out nearest-neighbor selector has only 53.1% exact winner accuracy, yet reaches practical equivalence to the oracle on substantially more requests. This difference confirms that exact label accuracy is a poor primary metric when several gate-passing experts have near-equal complete cost.

The 5×5 source-to- 6×6 held-out shift preserves the selected expert, but not every timing relation. Among the eight experts that pass the original-equation gate in both families, cost-rank Spearman correlation is 0.976 and maximum acceptance-probability calibration error is 0.020. The source-selected expert is $1.048 \times$ the fastest gate-passing held-out expert by competition median. If structurally ineligible experts are included before filtering, maximum calibration error instead reaches 0.9999. Thus structural eligibility is a prerequisite for meaningful probability calibration, and preserved winner identity does not imply zero complete-cost regret under size shift.

Without refitting, the conditioning/topology matrix retains a minimum paired-speedup 95% lower bound of 4.013, maximum filtered calibration error 0.020, and maximum selected-cost regret $1.052 \times$. Topology changes 5 expert gate statuses with zero mismatch or dangerous misroute. Since the embedding remains identical, capability/gate filtering—not embedding similarity alone—preserves selection; family and hardware transfer remain untested.

In the held-out joint-policy study, quadratic and cubic profiles select budgets 2 and 4. Across 64 disjoint scenarios, maximum oracle regret is $1.000 \times$, maximum pass-

TABLE 5

Final SuiteSparse routing outcomes. Lower regret is better. The v5 control-aware value is an offline replay over frozen observations (R); all other entries are recorded first-run policies or controls.

Policy	V5	V6
Current control-aware anchor	4.490 ^R	1.000030794
Raw conditioned model	2.457077	1.045287
Training family-fixed	4.489972	1.000030794
Global fixed SuperLU	2.613	1.001123
Static profile	2.393	1.756465

probability calibration difference is 0.000, and all requests succeed with 0 gate mismatches. The 32 first-stage rejections continue to 32 second-stage acceptances. These controlled families test profile transfer, not a learned universal policy.

Across 12 actions and 192 held-out requests, median cost error is 0.090; pass Brier/ECE are 0.0078/0.0177. Conditioned regret is $1.019 \times$ versus $1.330 \times$ for static and $1.330 \times$ for fixed controls. The router forms 45 plans, changes from the mode on 92.7% of requests, matches exhaustive enumeration, and records 0 terminal continuations and 0 gate mismatches. This controlled result does not establish public-workload dominance.

Table 5 preserves both final SuiteSparse outcomes. V5 uses immutable first-run observations; only its control-aware row is a zero-execution counterfactual. V6 is the unique prefrozen first run.

V5 falsifies a general repair claim: the current rule switches 0/24 requests and remains $1.718 \times$ global fixed. One family already shares the global anchor; another harmful family action passes both cost and support tests. This is within-support cost miscalibration, not gate failure.

V6 is favorable but narrow. Over 3 matrices, 24 requests, and five repetitions, regret is $1.000030794 \times$, 0.109% below fixed and 43.1% below static. It ties family-fixed because no specialization, adaptation, or transition is enabled. The guard removes the raw model’s $1.045287 \times$ excess; RHS-only and tolerance-only controls also tie the anchor, so fine-grained request adaptation is not established.

Median/p95/max cost error is 0.984/ 1.379/2.179 and Brier/ECE is 0.417/0.459, weak diagnostics despite near-oracle control. All 24 requests succeed with 8 terminal continuations and zero failure, gate, order, or DP mismatch. Interaction/order deltas of 0.046/0.092 remain diagnostic; no transition enters the frozen policy.

The nonlinear cascade supplies a counterexample to routing-as-free-speedup. On one coupled family, the online Equation-MoE path is $0.975 \times$ the classic baseline with a confidence interval below one; on a cubic family it reaches

1.199 $\times$. The router therefore needs workload qualification and negative outcomes, not only aggregate win rates.

7.4 RQ4: Learned Candidates, Correction, and Break-Even

Table 6 compares the two held-out operator studies. The linear study accepts all 6,400 requests with zero continuation or failure and achieves $1.078\times [1.070, 1.086]$ paired online speedup. Its measured training cost breaks even after 405 queries and remains amortized-positive at the 10,000-query projection.

The nonlinear operator’s raw candidate is dramatically inaccurate: its maximum normalized full-state and QoI errors are 597.5 and 469.3. The strict gate rejects the raw candidate. Generic Newton correction then converts all 6,400 requests into accepted same-accuracy solves and yields $1.430\times$ online speedup with $[1.413, 1.453]$. A weighted diagonal residual/Jacobi corrector remains at 0% acceptance after 32 steps, showing that correction is not a fungible post-processing layer.

The production-corrector frontier further separates the two families. The linear candidate already reaches full acceptance at budget 0, so extra correction is unnecessary on that family. For the nonlinear family, budget zero accepts no request and budget one accepts only 1.56%; the minimum full-acceptance budget is 2. At that operating point, candidate-inclusive complete cost is $0.623\times$ the budget-zero path, because avoiding the terminal classical solve more than pays for correction. All sweep points complete without Runtime failure. These family-specific thresholds do not imply one globally optimal correction budget.

The shared hybrid control makes this comparison end-to-end. On the linear family it accepts 6400 requests but reaches only $0.898\times$ versus the classical baseline; the verified runtime is $1.194\times [1.183, 1.210]$ faster. On the nonlinear family, the same control rejects all candidates, executes 6400 mandatory terminal continuations, and reaches only $0.561\times$; the verified runtime is $2.536\times [2.481, 2.570]$ faster. Both controls have zero failures and gate mismatches. Thus the gain is not explained by merely adding any correction-and-continuation wrapper to the learned candidate.

The native comparison executes official HINTS code, DeepONet, weights, and all 750 public cases; both methods have zero failures and maximum common residual 1.080×10^{-13} . The default Router chooses structured-tridiagonal-direct-cpu-v1 rather than forcing a learned path and is $14.059\times [13.506, 14.472]$ faster online; setup-amortized speedup over 10,000 requests is $14.216\times$. This supports complete-cost selection only for the official 1D Poisson workload, not a claim that a SMAVE latent operator beats HINTS or that the result transfers.

7.5 RQ5: Intra-node Scaling and Heterogeneous Placement

Figure 4 separates scaling of the original-equation gate from the complete verified path. At ten workers, the gate reaches $2.068\times$ paired speedup with a bootstrap 95% interval of $[1.982, 2.172]$, whereas the complete path reaches only $1.207\times$ with $[1.100, 1.354]$. Four workers reach $1.179\times$ complete-path speedup with $[1.085, 1.241]$; the two-worker

TABLE 6
Held-out operator studies. Speedup includes candidate construction, correction, gate, and required runtime work.

Metric	Linear	Nonlinear
Eval. requests	6,400	6,400
Accept/continue/failure	6,400/0/0	6,400/0/0
Validation cases	64	64
95% error-rate upper bound	4.57%	4.57%
Online speedup	$1.078\times$	$1.430\times$
95% interval	$[1.070, 1.086]$	$[1.413, 1.453]$
Shared-control speedup	$0.898\times$	$0.561\times$
Training wall time	10.24 ms	9.65 ms
Break-even queries	405	102
Max. state error	1.61×10^{-5}	7.29×10^{-5}
Raw candidate accepted	yes	no

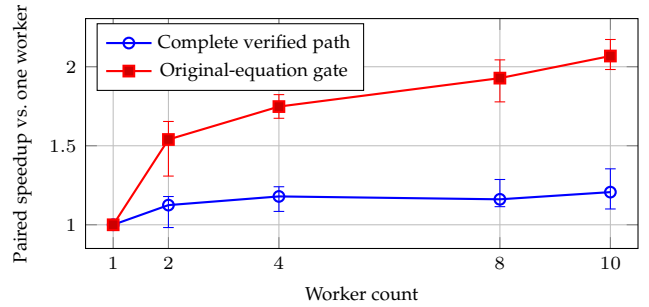


Fig. 4. Intra-node worker scaling on 40 incompressible Navier–Stokes Helmholtz solves over 30 repetitions. Error bars are fixed-seed bootstrap 95% intervals of paired speedup relative to one worker. Gate parallelism is useful, but the serial setup and spectral kernel bound complete-path gain.

interval still includes one. Across all runs, the maximum original-equation relative residual is 4.01×10^{-16} and the maximum cross-solver relative error is 1.37×10^{-10} .

The gate result demonstrates a concrete parallel verification mechanism, while the end-to-end result prevents it from being advertised as linear solver scaling. Classical PCG reaches $3.69\times$ paired speedup at ten workers in the same study, so the remaining SMAVE bottleneck is the worker-independent spectral kernel and setup rather than insufficient baseline parallelism. This is intra-node evidence on one Apple M4 host, not a multi-node or cross-architecture result.

The gate-only study broadens the mechanism beyond the Navier–Stokes stencil. Table 7 reports paired speedup over the same sequential fused gate. At ten workers, the linear family reaches $3.514\times$ with $[3.295, 3.905]$ and the nonlinear family reaches $2.403\times$ with $[2.171, 2.458]$. Across 30 repetitions of 2,080 requests per family, decisions and residuals exactly match the sequential gate. The different linear and nonlinear scaling curves show that gate parallelism depends on arithmetic intensity and per-request work; it is not a universal substitute for optimizing candidate generation or correction.

Table 8 tests amortization at a fixed ten requested workers. From 12 to 160 real-data solves, complete-path per-solve time falls from $19.44\mu s$ to $7.89\mu s$. Normalized complete throughput reaches $2.432\times$ with a bootstrap 95% interval of $[2.256, 2.521]$; gate throughput reaches $2.427\times$

TABLE 7

Gate-only worker scaling across two equation families. Values are paired median speedups over one worker; all decision and residual mismatch counts are zero.

Workers	Linear gate	Nonlinear gate
1	1.000×	1.000×
2	1.941×	1.590×
4	2.394×	1.797×
8	3.390×	2.144×
10	3.514×	2.403×

TABLE 8

Fixed-worker batch scaling, normalized to 12 solves. Intervals are from 30 paired repetitions; time is complete verified time per solve.

Solves	Time/solve (μ s)	Throughput scaling [95% CI]
12	19.44	1.000×
20	13.70	1.337× [1.246, 1.449]
40	10.71	1.741× [1.609, 1.903]
80	8.59	2.224× [2.104, 2.322]
160	7.89	2.432× [2.256, 2.521]

with [2.168, 2.725]. The maximum original-equation relative residual remains 5.71×10^{-16} and the maximum cross-solver relative error is 3.40×10^{-10} .

The batch curve confirms useful amortization but also saturates: a $13.3\times$ increase in request count yields only $2.43\times$ throughput scaling. Together with Figure 4, this identifies verification and setup/kernel fractions as different resource-allocation targets rather than one generic “parallel solver” effect.

The generic device execution probe runs real Metal and Apple Neural Engine candidates, verifies device/reference agreement, and accepts 64 GPU and two NPU requests without continuation. However, PDEBench-specific probes show that device backend support is not sufficient for performance. For Advection and Shallow Water, Metal and ANE candidates take tens of milliseconds and fail the final 10^{-10} original-equation gate, while CPU FP64 structured paths complete in microseconds. Burgers and Diffusion-Sorption device iterations similarly fail residual or complete-cost criteria. The router therefore rejects those device paths.

The second linear operator replication is another important negative result. Its paired bootstrap interval crosses one, so it does not establish stable acceleration despite retaining same-accuracy behavior. This prevents a positive Poisson-family result from being generalized to a different sparse topology.

Together, the negative paths validate the core design choice: heterogeneous execution must be selected by verified end-to-end cost. A system that reports only successful neural inference or forced accelerator kernels would reach the opposite and incorrect conclusion on these workloads.

8 ABLATION AND COST ANALYSIS

8.1 Component-Level Ablation

On a four-stage synthetic contract, Eq. (4) matches the minimum over all 24 permutations. This checks fixed-cascade ordering, not probability calibration or globally optimal subset selection.

The component matrix isolates router, candidate, correction, gate, and continuation on one 64-scenario linear workload. Forcing the operator while retaining correction and gate separates expert quality from router quality. A finite but deliberately bad candidate is rejected and reaches the deterministic terminal solver after the configured correction limit. Tampered expert, certificate, and RuntimeBundle artifacts are rejected by the production solve path rather than only by an offline checker.

Removing the gate would make the nonlinear operator appear orders of magnitude faster because raw inference takes approximately 0.58μ s, but the maximum normalized state error exceeds 597. Removing correction would therefore convert a profitable verified path into a fast wrong answer. Conversely, applying the wrong corrector preserves the acceptance test but yields 0% acceptance and no acceleration. These two ablations distinguish correctness from usefulness.

The correction-budget sweep exposes a second mechanism-level ablation. On the linear family, budget zero already passes every original-equation test. On the nonlinear family, moving from zero to one correction step raises acceptance only to 1.56%, whereas two steps reach full acceptance and reduce candidate-inclusive complete cost to $0.623\times$ the zero-step path. The relevant frontier is therefore discrete and family-dependent: too little correction pays for rejection and classical continuation, while unnecessary correction adds cost after acceptance has saturated. A production profile–Router–Newton probe confirms the mechanism: the same nonlinear candidate continues to the terminal solver at budget zero but is accepted after two Newton steps at budget two; this is not a universal two-step optimum.

For three actions over two experts, the joint DP selects $(e_B, 1) \rightarrow (e_A, 2)$ at 4.5μ s, matches the exhaustive oracle, costs $0.900\times$ the best single action, and executes the gate-verified rejection–continuation–acceptance path. Both fixed-seed 256-case independent and 256-case interaction sweeps have zero oracle gap; interactions change 126 plans, and all 4096 graphs satisfy Proposition 1. Planning-only states/transitions are 49/204 for the deployed 12-action, $k = 3$ shape and 158,209/1,412,192 for the largest executed 32-action, $k = 6$ profile. Under the one-million cap, exact preflight admits 40 actions at 666,561 states but rejects 44 at 1,227,425 before any DP-state visit. These are optimizer/control checks, not transfer, timing, or workload-speedup evidence.

8.2 Fusing a Strict Gate

Reorganizing the same FP64 original-expression calculation yields $1.754\times$ and $1.271\times$ gate speedup over 100 repetitions of 65 requests, with lower bounds 1.738 and 1.272 and identical decisions/residuals. Verification can therefore be optimized without weakening acceptance, although full-solve impact remains workload-dependent because candidate and corrector cost can dominate.

8.3 Router Regret and Model Complexity

More complex selectors do not automatically improve deployable routing. The nearest-neighbor and depth-3 CART

baselines narrow practical regret, but winner labels and selected experts vary with microsecond timing noise; a structure-derived selector also changes rank across runs. We therefore report regret, not winner accuracy, and retain a gate-filtered top- k cascade. Control-aware abstention reduces v6 from raw $1.045287\times$ to $1.000030794\times$, but switches no frozen v5 request; v6 alone would overstate robustness.

8.4 Where Time Goes

Raw candidates are only 1.60% and 0.26% of layered full-path time; correction plus Runtime gate accounts for 98.45% and 99.74%. Generic correction reaches only $0.898\times/0.561\times$, versus $1.078\times/1.430\times$ for complete paths; specialized recurrence, FFT, SIMD, and batched gates also drive PDEBench gains. Thus the target is fusion, not inference alone.

9 DISCUSSION AND LIMITATIONS

V6 nearly matches oracle abstention but improves fixed by only 0.109%; frozen v5 switches no request and remains $1.718\times$ fixed. Brier/ECE 0.417/0.459 and p95 cost error 1.379 preclude calibrated-model claims. Solver selection and components are prior art; the contribution is gate-controlled verified complete-path composition.

External validity is narrow: all timing uses one Apple M4, and the PDEBench-derived, resource-limited COPS, one-HINTS, and three-matrix/24-request v5/v6 studies establish neither architecture nor population performance; opposite v5/v6 outcomes show mixed validity. Multi-host validation was attempted on a second ARM64 host (Nvidia DGX Station) but could not complete due to 40–60 $\times$ performance penalty on large matrices; SMAVE build and execution succeeded, but platform suitability for timing benchmarks varies. A post-hoc zero-execution audit finds 32 development-supported pairs, all absent at unguarded top-3 selection. Five different unsupported candidates appear across 17 v6 requests, but the control-aware gate removes all; final candidates and timings are zero. Development-held-out overlap is zero (v5/v6 held-out sets 8/56, Jaccard 0.143), so no multiplier or interaction benefit is inferred. One PDEBench-derived case exceeds 100 $\times$; eight SuiteSparse cases lack common success; device, nonlinear, topology, and order effects remain. Broader prefrozen cohorts, architectures, and independent reruns are required.

Residual gates cannot detect wrong callbacks, faulty arithmetic, or unavailable truth; small residual may mask state error in ill-conditioned systems [16]. Claims are only to declared equation/constraint tolerances. The interaction optimizer models adjacent costs, not longer-memory costs or history-dependent acceptance; general high-index reduction is excluded.

10 CONCLUSION

We formulate exact finite selection over candidate, correction, gate, continuation, and terminal cost, including an NP-complete adjacent-interaction model. V6 reaches $1.000030794\times$ oracle regret and 0.109% below fixed with zero correctness, order, or DP mismatch; frozen v5 switches

no request and remains $1.718\times$ fixed. Together with retained regressions, this supports workload-qualified verified cascade composition—not solver-selection novelty, general robustness, or universal speedup.

REFERENCES

- [1] Y. Saad and M. H. Schultz, “GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems,” *SIAM Journal on Scientific and Statistical Computing*, vol. 7, no. 3, pp. 856–869, 1986.
- [2] X. S. Li, “An overview of SuperLU: Algorithms, implementation, and user interface,” *ACM Transactions on Mathematical Software*, vol. 31, no. 3, pp. 302–325, 2005.
- [3] A. C. Hindmarsh, P. N. Brown, K. E. Grant, S. L. Lee, R. Serban, D. E. Shumaker, and C. S. Woodward, “SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers,” *ACM Transactions on Mathematical Software*, vol. 31, no. 3, pp. 363–396, 2005.
- [4] P. R. Amestoy, I. S. Duff, J.-Y. L’Excellent, and J. Koster, “A fully asynchronous multifrontal solver using distributed dynamic scheduling,” *SIAM Journal on Matrix Analysis and Applications*, vol. 23, no. 1, pp. 15–41, 2001.
- [5] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier neural operator for parametric partial differential equations,” in *International Conference on Learning Representations*, 2021.
- [6] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.
- [7] Y. Li, P. Y. Chen, T. Du, and W. Matusik, “Learning preconditioners for conjugate gradient PDE solvers,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 19 425–19 439.
- [8] J. R. Rice, “The algorithm selection problem,” in *Advances in Computers*. Elsevier, 1976, vol. 15, pp. 65–118.
- [9] L. Kotthoff, “Algorithm selection for combinatorial search problems: A survey,” *AI Magazine*, vol. 35, no. 3, pp. 48–60, 2014.
- [10] E. R. Jessup, P. Motter, B. Norris, and K. Sood, “Performance-based numerical solver selection in the lighthouse framework,” *SIAM Journal on Scientific Computing*, vol. 38, no. 5, pp. S750–S771, 2016.
- [11] S. Balay, W. D. Gropp, L. C. McInnes, and B. F. Smith, “Efficient management of parallelism in object-oriented numerical software libraries,” in *Modern Software Tools for Scientific Computing*. Birkhäuser, 1997, pp. 163–202.
- [12] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, “PDEBench: An extensive benchmark for scientific machine learning,” in *Advances in Neural Information Processing Systems: Datasets and Benchmarks Track*, 2022.
- [13] A. Rudikov, V. Fanaskov, E. Muravleva, Y. M. Laevsky, and I. Osleledets, “Neural operators meet conjugate gradients: The FCG-NO method for efficient PDE solving,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 42 766–42 782.
- [14] E. Zhang, A. Kahana, A. Kopaničáková, E. Turkel, R. Ranade, J. Pathak, and G. E. Karniadakis, “Blending neural operators and relaxation methods in PDE numerical solvers,” *Nature Machine Intelligence*, vol. 6, pp. 1303–1313, 2024.
- [15] S. Rayan, Y. Patel, and A. Tewari, “A greedy PDE router for blending neural operators and classical methods,” *arXiv preprint arXiv:2509.24814*, 2025.
- [16] H. Jiang, L. Wang, P.-C. Chen, M. S. Kwak, S. Kim, B. Bell, and J. J. Park, “Error-conditioned neural solvers,” *arXiv preprint arXiv:2606.27354*, 2026.
- [17] K. Zhang, D. Chang, M. Yang, J. Zhao, and W. Yang, “SPECTRA: Revitalizing image-based iterative method selection for sparse linear systems,” *Expert Systems with Applications*, vol. 319, p. 132050, 2026.
- [18] Y. Zabegaev, I. Berre, and E. Keilegavlen, “Data-driven linear solver selection and performance tuning for multiphysics simulations in porous media,” *Computational Science and Engineering*, vol. 3, p. 4, 2026.
- [19] M. Khodak, E. Chow, M.-F. Balcan, and A. Talwalkar, “Learning to relax: Setting solver parameters across a sequence of linear system instances,” in *International Conference on Learning Representations*, 2024.

- [20] D. Berend, R. Brafman, S. Cohen, S. E. Shimony, and S. Zucker, "Optimal ordering of independent tests with precedence constraints," *Discrete Applied Mathematics*, vol. 162, pp. 115–127, 2014.
- [21] S. Ruan, F. Tu, K. R. Pattipati, and A. Patterson-Hine, "On a multimode test sequencing problem," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 34, no. 3, pp. 1490–1499, 2004.